

직접 DB에 저장하고 꺼내 쓰는 RAG 챗봇

TXT/PDF → chunk → embedding → PostgreSQL/pgvector → 검색 → context → 답변

최종 결과물: CLI에서 동작하는 문서 기반 RAG 챗봇



오늘 완성할 결과물

저장·검색·답변이 이어지는 CLI RAG 챗봇을 만듭니다.

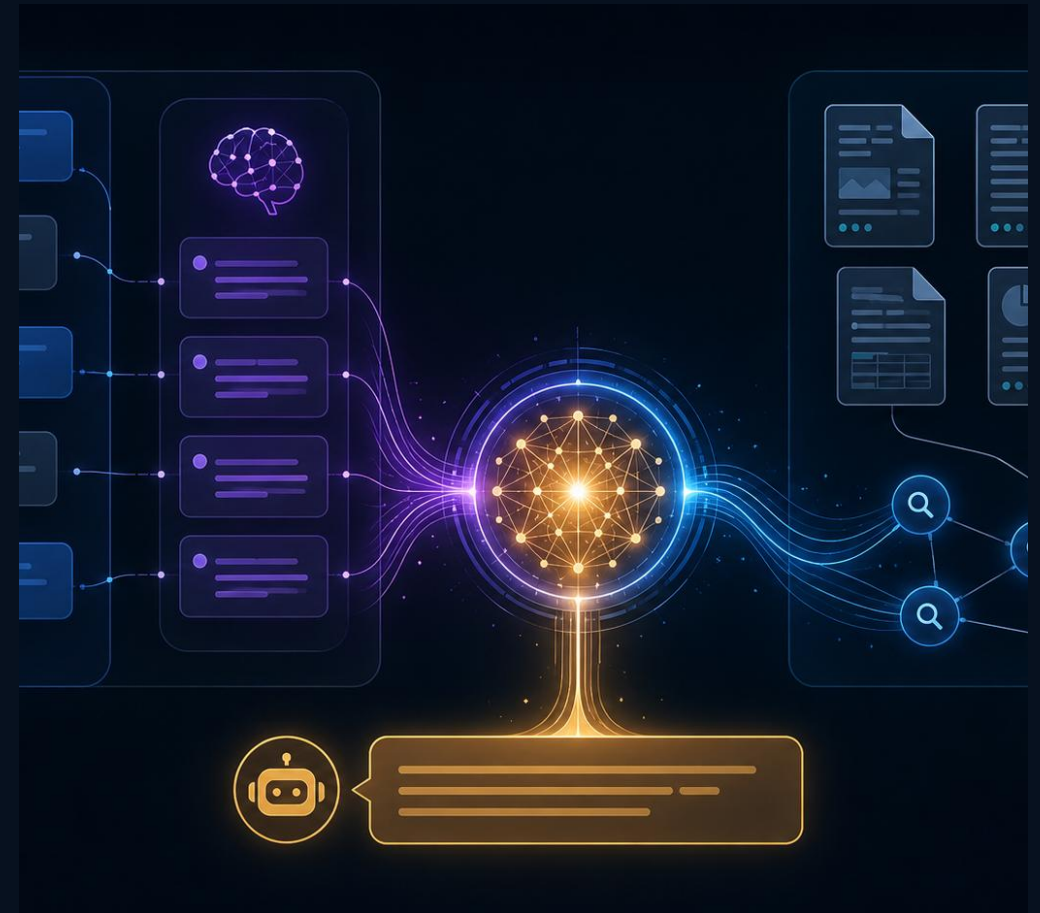
TXT/PDF를 chunk로 나누고 embedding을 DB에 저장
사용자 질문을 embedding해서 가까운 chunk 검색
검색된 chunk를 context로 넣어 LLM 답변 생성
CLI에서 질문을 반복 입력받고 출처를 함께 출력

질문 > 중국 AI 플러스 정책의 핵심 영역은?

답변
중국 AI 플러스 정책은 ...

[검색된 출처]

1. SPRI AI Brief_10월호_산업동향_1002_F.pdf / p.4 / distance=0.23



2일차와 3일차의 연결

어제의 embedding을 오늘은 저장 가능한 검색 자산으로 바꿉니다.

오해 / 문제

2일차: embedding을 만들어보고 길이와 값을 확인
문제: 프로그램이 끝나면 vector도 함께 사라짐

정확한 구조

3일차: embedding을 DB에 저장하고 이후 질문에
서 재사용
핵심 변화: “숫자 만들기” 에서 “검색 가능한 자
산 만들기” 로 이동

```
# 2일차: embedding 확인
vector = embeddings.embed_query("질문")
print(len(vector))

# 3일차: embedding 저장
INSERT INTO rag_embeddings
(chunk_id, embedding_model, embedding)
VALUES (... , ..., %s::vector)
```

실습 파일 전체 흐름

파일 이름이 곧 수업 흐름입니다. 위에서 아래로 실행하면 챗봇이 완성됩니다.

00 00_check_env_and_db.py

01 01_preview_data.py

02 02_insert_txt_embeddings.py

03 03_search_txt_from_db.py

04 04_insert_pdf_embeddings.py

05 05_sql_vector_search.py

06 06_basic_rag_once.py

07 07_cli_rag_chatbot.py

08 assignment_day3_cli_rag_chatbot_starter.py

번호 순서대로 실행하면 저장 → 검색 → 답변 → CLI 챗봇 흐름이 완성됩니다.

오늘 다루는 데이터

작은 TXT로 먼저 성공시키고, 같은 흐름을 PDF에 확장합니다.

Sample_Text.txt: LangChain, LLM, RAG,
LCEL을 설명하는 짧은 텍스트

SPRi AI Brief_10월호_산업동향_1002_F.pdf: 정
책·기업·기술·교육 질문 실습용 PDF

데이터 내용 암기가 아니라 저장 구조와 검색 흐름 이해가 목적

RAG는 “ PDF 통째로 넣기” 가 아니다

RAG는 검색과 생성을 분리해서 다루는 구조입니다.

오해 / 문제

잘못된 이해: PDF 전체를 LLM에게 그대로 넣는다
문제: 길이 제한, 비용 증가, 관련 없는 내용 포함,
근거 확인 어려움

정확한 구조

정확한 이해: 문서를 chunk로 나누고, 질문과 가까운 chunk만 검색한다
LLM은 문서를 직접 찾아보지 않는다. 우리가 찾아서 넣어줘야 한다.

전체 파이프라인

오늘 작성하는 코드는 이 한 장의 흐름을 파일별로 나눈 것입니다.



저장 단계: 문서는 한 번 embedding해서 DB에 저장합니다.

질문 단계: 사용자 질문은 매번 embedding해서 DB의 vector와 비교합니다.

문서 chunk vector ← 미리 DB에 저장 | 질문 vector ← 질문이 들어올 때 생성

저장 단계와 질문 단계는 다르다

문서는 미리 저장하고, 질문은 들어올 때마다 embedding합니다.

문서 저장 단계: 여러 chunk를 `embed_documents()`로 embedding

질문 검색 단계: 사용자 질문 하나를 `embed_query()`로 embedding

- 문서 vector는 DB에 남아 있고 재사용된다
- 질문 vector는 검색 순간에만 필요하다

```
# 문서 저장 단계
vectors =
embeddings.embed_documents(chunk_texts)

# 질문 검색 단계
query_vector =
embeddings.embed_query(question)
```


.env는 왜 필요한가

API Key와 DB 접속 정보는 코드가 아니라 환경변수로 관리합니다.

OPENAI_API_KEY: OpenAI 호출에 필요한 키
OPENAI_MODEL: 답변 생성에 사용할 chat model
OPENAI_EMBEDDING_MODEL: chunk와 질문을
vector로 바꿀 embedding model
POSTGRES_*: DB 접속 정보
MAX_CHUNKS_FOR_PRACTICE: PDF 일부만 먼저
테스트할 때 사용

```
OPENAI_API_KEY=...  
OPENAI_MODEL=gpt-4o-mini  
OPENAI_EMBEDDING_MODEL=text-embedding-3-small
```

```
POSTGRES_HOST=localhost  
POSTGRES_PORT=5432  
POSTGRES_DB=testdb  
POSTGRES_USER=test  
POSTGRES_PASSWORD=...
```

```
MAX_CHUNKS_FOR_PRACTICE=10
```

00_check_env_and_db.py

시작 전에 환경 오류를 막는 안전장치입니다.

.env 값이 들어왔는지 확인
PostgreSQL 연결 확인
pgvector extension 확인
rag_documents, rag_chunks, rag_embeddings 테이블
확인
여기서 실패하면 뒤 파일로 넘어가지 않는다

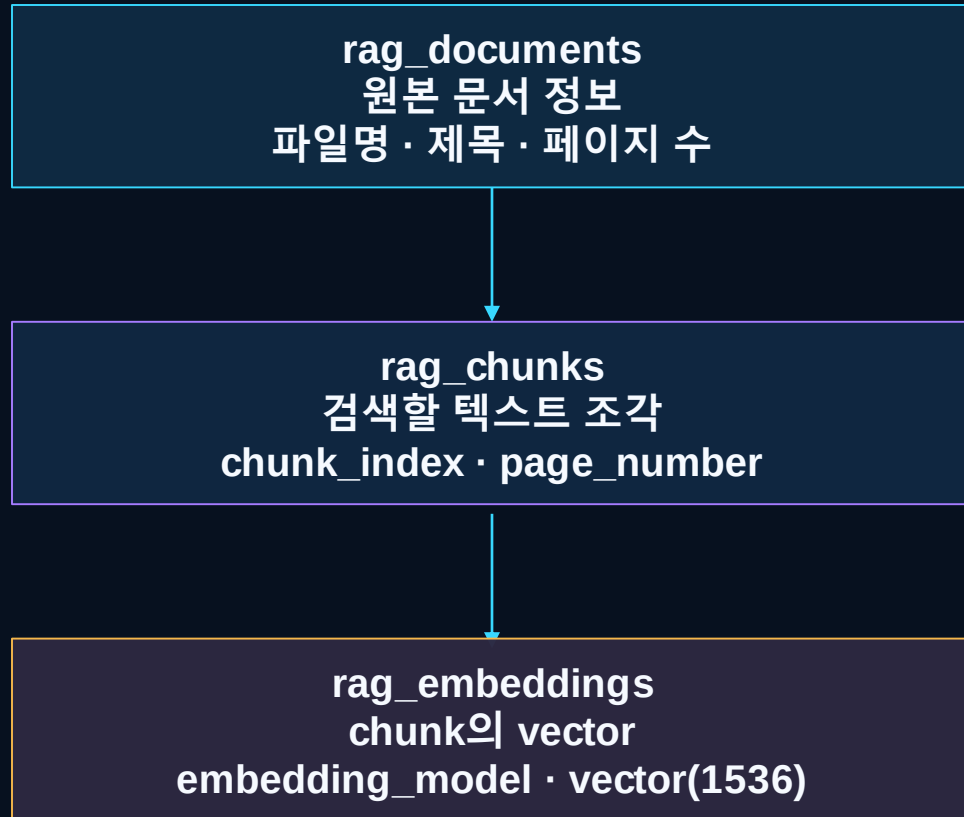
```
python 00_check_env_and_db.py
```

```
[1] .env 확인  
[2] DB 연결 확인  
[3] 실습 테이블 확인
```

실행 → 출력 해석 → 핵심 코드 확인 → 다음 단계

DB 테이블은 세 개만 본다

초심자 버전에서는 document, chunk, embedding 구조만 확실히 잡습니다.



```
rag_documents.id
  ↓ document_id
rag_chunks.id
  ↓ chunk_id
rag_embeddings.embedding = [0.012, -0.032, ...]
```

검색 대상은 PDF 파일이 아니라
rag_chunks.content와
rag_embeddings.embedding입니다.

JOIN으로 답변 근거를 다시 조립한다

vector 검색 결과는 chunk, 문서 정보와 다시 연결되어야 합니다.

embedding 거리 계산은 rag_embeddings에서 일어남
답변 context는 rag_chunks.content에서 가져옴
출처 표시는 rag_documents.file_name과
rag_chunks.page_number 사용
세 테이블을 JOIN해야 검색 결과가 사람이 읽을 수 있는 형태가 됨

```
SELECT
  d.file_name,
  c.page_number,
  c.content,
  e.embedding <=> %s::vector AS distance
FROM rag_embeddings e
JOIN rag_chunks c ON e.chunk_id = c.id
JOIN rag_documents d ON c.document_id = d.id
ORDER BY distance
LIMIT %s;
```

01_preview_data.py

RAG는 데이터를 먼저 눈으로 확인해야 합니다.

TXT 파일 크기와 앞부분 확인
PDF 페이지 수 확인
PyPDFLoader로 page_content 일부 출력
텍스트가 비어 있거나 깨지면 embedding 전에 원인을
찾아야 함

```
python 01_preview_data.py

loader = PyPDFLoader(pdf_name)
pages = loader.load()

print(len(pages))
print(pages[0].page_content[:500])
```

실행 → 출력 해석 → 핵심 코드 확인 → 다음 단계

Document는 문자열 하나가 아니다

LangChain의 Document는 본문과 부가 정보를 함께 들고 있습니다.

page_content: LLM context로 들어갈 실제 본문
metadata: source, page 같은 부가 정보
본문 검색과 출처 표시는 서로 다른 필드에서 나온다
RAG 챗봇은 답변 아래에 file_name, page_number,
distance를 출력

```
Document(  
  page_content="중국 국무원, AI 플러스...",  
  metadata={  
    "source": "SPRi AI Brief_...pdf",  
    "page": 3  
  }  
)
```

왜 chunk로 나누는가

검색은 문서 전체가 아니라 관련 있는 조각을 찾는 작업입니다.

- 길이 제한: 긴 문서 전체를 한 번에 처리하기 어렵다
- 검색 정확도: 문서 전체 vector는 주제가 너무 넓다
- 답변 근거: chunk 단위로 page와 문단 근거를 보여주기 쉽다
- 비용 관리: 필요한 조각만 LLM context로 넣는다

RecursiveCharacterTextSplitter 읽는 법

chunk_size, chunk_overlap만 정확히 이해해도 충분합니다.

chunk_size: chunk 하나의 최대 길이
chunk_overlap: 앞뒤 chunk가 겹치는 길이
overlap은 문맥이 자르는 지점에서 끊기는 문제를 줄인다
초심자 버전에서는 깊은 튜닝보다 개념 이해가 우선

```
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=800,  
    chunk_overlap=120,  
)
```

```
chunks = splitter.split_documents(pages)
```

embedding은 의미를 숫자 좌표로 바꾸는 과정

의미가 비슷한 텍스트는 가까운 vector가 되도록 표현합니다.

텍스트를 고정 길이 숫자 리스트로 변환
동일한 embedding 모델로 문서와 질문을 변환해야 비교 가능
exact keyword match가 아니라 semantic search를 가능하게 함
오늘은 text-embedding-3-small 기준 vector(1536)를 사용



embed_documents vs embed_query

문서 저장용 함수와 질문 검색용 함수를 구분합니다.

embed_documents(): 여러 chunk를 한 번에
embedding

embed_query(): 질문 하나를 검색용 vector로 변환
문서 chunk는 미리 저장
사용자 질문은 매번 embedding

```
# 02, 04번 파일: 문서 저장
vectors = embeddings.embed_documents(texts)

# 03, 05번 파일: 질문 검색
query_vector = embeddings.embed_query(question)
```

pgvector의 핵심: cosine distance

이 실습에서는 낮을수록 더 비슷한 distance로 해석합니다.

<=> 연산자는 cosine distance 계산에 사용
distance는 0에 가까울수록 더 비슷함
ORDER BY distance ASC로 가까운 chunk를 먼저 가져옴
유사도 점수와 distance를 혼동하지 않기

```
ORDER BY e.embedding <=> %s::vector  
LIMIT %s
```

```
-- cosine similarity가 필요하다면  
-- 1 - (embedding <=> query_vector)
```

02_insert_txt_embeddings.py

작은 TXT로 document → chunk → embedding → DB 저장 흐름을 먼저 성공시킵니다.

Sample_Text.txt 읽기
chunk_size=300, chunk_overlap=50으로 분할
embed_documents()로 chunk별 vector 생성
rag_documents → rag_chunks → rag_embeddings 순
서로 저장

```
python 02_insert_txt_embeddings.py  
  
document_id = INSERT rag_documents  
chunk_id    = INSERT rag_chunks  
             INSERT rag_embeddings
```

실행 → 출력 해석 → 핵심 코드 확인 → 다음 단계

DB에서 직접 확인한다

저장했다고 끝내지 말고 SELECT로 실제 row를 확인합니다.

원본 문서 row가 들어갔는지 확인
chunk 개수와 preview 확인
embedding 개수가 chunk 개수와 맞는지 확인
print가 아니라 DB 결과로 저장을 검증

```
SELECT d.title, c.chunk_index, LEFT(c.content, 100) AS preview
FROM rag_documents d
JOIN rag_chunks c ON d.id = c.document_id
WHERE d.file_type = 'txt'
ORDER BY c.chunk_index;
```

```
SELECT embedding_model, COUNT(*)
FROM rag_embeddings
GROUP BY embedding_model;
```

03_search_txt_from_db.py

질문으로 이미 저장된 TXT chunk를 다시 꺼내옵니다.

질문을 embed_query()로 vector 변환
DB의 chunk vector와 cosine distance 계산
가까운 chunk TOP_K개 반환
문서는 다시 embedding하지 않는다

```
python 03_search_txt_from_db.py

QUESTION = "LangChain에서 RAG는 왜 필요한가요?"
query_vector =
embeddings.embed_query(QUESTION)
results = search_similar_chunks(..., top_k=3)
```

실행 → 출력 해석 → 핵심 코드 확인 → 다음 단계

TOP_K는 답변 품질에 영향을 준다

가져오는 chunk 수가 많다고 항상 좋은 것은 아닙니다.

TOP_K가 작으면 필요한 근거가 부족할 수 있음
TOP_K가 크면 관련 없는 chunk가 섞일 수 있음
context가 길어지면 비용과 혼란이 증가
질문 성격과 chunk 크기에 따라 실험 필요

```
TOP_K = 1  
TOP_K = 3  
TOP_K = 5
```

같은 질문으로 결과와 답변을 비교해보기

04_insert_pdf_embeddings.py

TXT에서 성공한 저장 흐름을 PDF로 확장합니다.

PyPDFLoader로 PDF를 page 단위 Document로 읽기
split_documents()로 PDF chunk 생성
embed_documents()로 chunk별 vector 생성
세 테이블에 동일한 구조로 저장

```
python 04_insert_pdf_embeddings.py
```

```
PyPDFLoader → pages
```

```
split_documents → chunks
```

```
embed_documents → vectors
```

```
INSERT → documents / chunks / embeddings
```

실행 → 출력 해석 → 핵심 코드 확인 → 다음 단계

PDF page metadata 처리

loader의 page와 사람이 보는 페이지 번호는 다를 수 있습니다.

PyPDFLoader metadata page는 보통 0부터 시작
사용자에게 보여줄 page_number는 +1 보정
출처 표시에는 사람이 읽는 페이지 번호가 필요
검색 품질뿐 아니라 검증 가능성도 설계해야 함

```
raw_page = doc.metadata.get("page")
page_number = raw_page + 1 if isinstance(raw_page, int) else None

INSERT INTO rag_chunks (... , page_number, content)
```

PDF embedding은 작게 성공시키고 확장한다

처음부터 전체 PDF를 돌리지 않아도 됩니다.

embedding은 API 호출이므로 시간과 비용이 든다
MAX_CHUNKS_FOR_PRACTICE=10으로 일부
chunk만 먼저 테스트
DB 저장과 검색이 성공하면 전체 적재로 확장
초심자 수업에서는 “ 작게 성공 → 확장 ” 이 안전하다

```
MAX_CHUNKS_FOR_PRACTICE=10
```

```
# 0이면 전체 chunk 사용  
# 10이면 앞의 10개 chunk만 사용
```

05_sql_vector_search.py

PDF에서 질문과 가까운 chunk를 찾습니다.

질문: 중국 AI 플러스 정책의 핵심 영역은?

질문 embedding 생성

PDF chunk embedding과 distance 계산

file_name, page_number, content preview, distance 확인

```
python 05_sql_vector_search.py
```

```
QUESTION = "중국 AI 플러스 정책의 핵심 영역은 무엇인가요?"
```

```
TOP_K = 4
```

실행 → 출력 해석 → 핵심 코드 확인 → 다음 단계

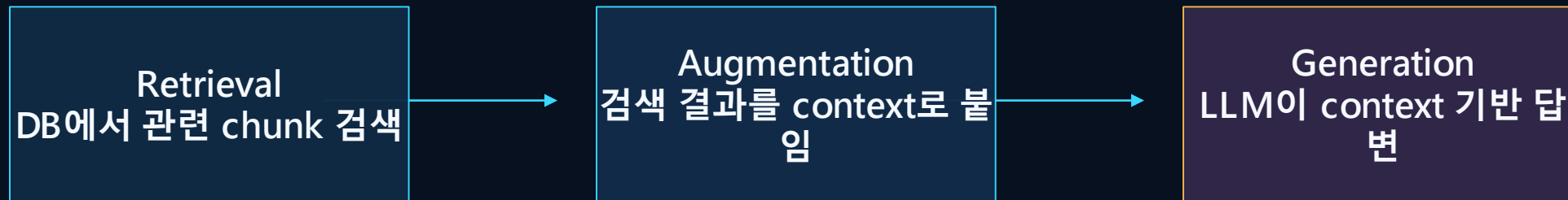
RAG 디버깅은 검색 결과부터 본다

답변이 이상하면 LLM 보다 먼저 content preview를 확인합니다.

검색된 chunk가 질문과 관련 있는가?
distance가 낮은 결과가 위에 있는가?
page_number와 file_name이 정상인가?
검색 결과가 틀리면 prompt를 고쳐도 한계가 있다

RAG의 3단계

Retrieval, Augmentation, Generation을 코드 흐름으로 연결합니다.



```
results = search_similar_chunks(...)
context = format_context(results)
response = llm.invoke([...])
```

Prompt에서 context를 제한한다

검색만 잘해도 프롬프트가 느슨하면 문서 밖 이야기를 할 수 있습니다.

제공된 context를 우선해서 답변하도록 지시
context에 없는 내용은 추측하지 말라고 명시
답변 언어와 형식을 지정
좋은 RAG 답변 = 좋은 검색 + 좋은 prompt

```
SYSTEM_PROMPT = """
```

```
너는 문서 기반으로 답변하는 RAG 챗봇입니다.
```

```
규칙:
```

1. 반드시 제공된 context를 우선해서 답변하세요.
2. context에 없는 내용은 추측하지 말고 모른다고 말하세요.
3. 답변은 한국어로 작성하세요.

```
"""
```

06_basic_rag_once.py

한 번의 질문으로 검색 → context → 답변 전체 흐름을 확인합니다.

검색 결과를 먼저 출력
검색 결과로 context 구성
LLM에게 context와 질문 전달
답변이 검색 결과와 맞는지 확인

```
python 06_basic_rag_once.py
```

```
QUESTION = "중국 AI 플러스 정책의 핵심 영  
역과 목표를 요약해줘."
```

실행 → 출력 해석 → 핵심 코드 확인 → 다음 단계

context는 LLM에게 보여주는 작업대다

검색 결과를 그대로 던지지 말고 읽기 좋은 문자열로 정리합니다.

각 검색 결과에 문서 번호와 출처 표시
file_name과 page_number를 함께 넣으면 근거 추적이
쉬움
content 사이에 구분선을 두면 LLM과 사람이 모두 읽
기 쉬움
context가 길어질수록 비용과 혼란이 증가할 수 있음

[문서 1 /SPRi AI Brief_...pdf / p.4]
중국 국무원은 ...

[문서 2 /SPRi AI Brief_...pdf / p.5]
AI 플러스 행동은 ...

07_cli_rag_chatbot.py

질문 하나짜리 RAG를 반복 입력 구조로 바꾸면 CLI 챗봇이 됩니다.

while True로 계속 질문 입력 받기
quit, exit, 종료 입력 시 break
질문마다 검색과 답변 생성 수행
답변 아래에 검색된 출처 출력

```
while True:
    question = input("질문 > ").strip()

    if question.lower() in {"quit", "exit"} or question == "종료":
        break

    answer, results = answer_question(question)
    print(answer)
    print_sources(results)
```

실행 → 출력 해석 → 핵심 코드 확인 → 다음 단계

CLI 챗봇의 함수 분리

반복문보다 중요한 것은 검색과 답변 생성을 함수로 나누는 것입니다.

retrieve: 질문으로 관련 chunk 검색
format_context: 검색 결과를 context로 정리
generate: LLM 답변 생성
print_sources: 출처 출력
함수 분리가 되어야 과제와 다음 수업으로 확장 가능

```
def answer_question(question):  
    results = retrieve_documents(question)  
    answer = generate_answer(question, results)  
    return answer, results
```

과제 TODO 1: retrieve_documents()

질문을 받아 DB에서 관련 chunk를 가져오는 함수입니다.

embedding 모델 준비
질문을 embed_query()로 vector 변환
rag_embeddings, rag_chunks, rag_documents JOIN 검색
results를 list[dict] 형태로 반환
이 함수는 답변을 만들지 않고 검색만 담당

```
def retrieve_documents(question):  
    embeddings = OpenAIEmbeddings(model=embedding_model)  
    query_vector = embeddings.embed_query(question)  
  
    # SQL: JOIN + ORDER BY e.embedding <=> query_vector  
    # return results
```

과제 TODO 2: generate_answer()

검색 결과를 context로 바꾸고 LLM에게 답변을 요청합니다.

format_context(results)로 context 생성
context와 question을 user_prompt에 넣기
SystemMessage와 HumanMessage로 LLM 호출
response.content 반환
이 함수는 DB 검색을 하지 않는다

```
def generate_answer(question, results):  
    context = format_context(results)  
    user_prompt = f"""  
아래 context만 참고해서 질문에 답변하세요.  
  
[context]  
{context}  
  
[질문]  
{question}  
"""  
    response = llm.invoke([...])  
    return response.content
```

과제 테스트 질문

답변만 보지 말고 출처와 distance까지 확인합니다.

중국 AI 플러스 정책의 핵심 영역은 무엇인가요?
오픈AI는 챗GPT 안전성 개선을 위해 어떤 계획을 발표했나요?
Qwen3-Next의 특징을 요약해줘.
교육 분야에서 AI는 어떻게 활용되고 있나요?
종료 명령: quit / exit / 종료

자주 나는 오류와 보는 위치

오류 메시지를 보고 어느 단계 문제인지 구분합니다.

OPENAI_API_KEY 오류: .env 확인

DB 연결 실패: host, port, user, password 확인

relation does not exist: day3_schema.sql 실행 여부 확인

vector dimension mismatch: embedding 모델과 vector(1536) 차원 확인

답변 근거 이상: 검색 결과 content preview 먼저 확인

RAG 품질 점검 순서

문제가 생겼을 때는 아래 순서로 확인합니다.

1. 문서가 제대로 로딩되었는가?
2. chunk가 적절한 크기로 나뉘었는가?
3. embedding이 chunk 개수만큼 저장되었는가?
4. 검색 결과가 질문과 관련 있는가?
5. prompt가 context 기준 답변을 제한하는가?

오늘 남겨야 하는 핵심 문장

학생들이 이 문장들을 말할 수 있으면 Day 3는 성공입니다.

- 1 PDF를 그대로 LLM에게 넣는 것이 아니라 chunk로 나눈다.
- 2 chunk마다 embedding을 만들어 DB에 저장한다.
- 3 사용자 질문도 embedding해서 DB의 vector와 비교한다.
- 4 가까운 chunk를 가져와 context로 만든다.
- 5 LLM은 그 context를 보고 답변한다.
- 6 RAG 답변에는 출처를 함께 보여주는 것이 좋다.

4일차로 이어지는 질문

오늘 만든 검색 함수를 내일은 Retriever 전략으로 확장합니다.

오늘: 질문 embedding과 chunk embedding의 거리로
검색하는 Dense 검색
다음: 키워드 기반 Sparse 검색, 의미 기반 Dense 검색,
Hybrid 검색 비교
검색 전략이 바뀌면 context가 바뀜
context가 바뀌면 LLM 답변도 바뀜

```
# Day 3
```

```
results = search_similar_chunks(...)
```

```
# Day 4
```

```
results = retrieve(query, retriever_type="dense")
```

```
results = retrieve(query, retriever_type="sparse")
```

```
results = retrieve(query, retriever_type="hybrid")
```

실행 파일명 모음

실습 중 현재 단계가 헛갈리면 이 장으로 돌아옵니다.

```
python 00_check_env_and_db.py
python 01_preview_data.py
python 02_insert_txt_embeddings.py
python 03_search_txt_from_db.py
python 04_insert_pdf_embeddings.py
python 05_sql_vector_search.py
python 06_basic_rag_once.py
python 07_cli_rag_chatbot.py
python assignment_day3_cli_rag_chatbot_starter.py
```

Day 3 마무리

저장 · 검색 · context · 답변

Store
chunk + embedding

Search
question vector

Context
retrieved chunks

Answer
grounded response

다음: Retriever를 바꾸면 검색 결과와 답변은 어떻게 달라질까?